

《计算机组成原理》实验报告

年级、专业、班级	2024 级计算机科技, 01 班, 03 班, 02 班		姓名	BROEKHOFROBIN, 严浩睿, 吴林兴
实验题目	实验二 处理器译码实验			
实验时间	4 月, 19 号	实验地点	DS1421	
实验成绩		实验性质	☐ 验证性 ☒ 设计性 ☐ 综合性	
教师评价: ☐算法/实验过程正确; ☐源程序/实验内容提交 ☐程序结构/实验步骤合理; ☐实验结果正确; ☐语法、语义正确; ☐报告规范; 其他: 评价教师签名:				
一、实验目的 (1) 掌握单周期 CPU 控制器的工作原理及其设计方法。 (2) 掌握单周期 CPU 各个控制信号的作用和生成过程。 (3) 掌握单周期 CPU 执行指令的过程。 (4) 掌握取指、译码阶段数据通路执行过程				
二、实验项目内容 1. PC。D 触发器结构, 用于储存 PC(一个周期)。需实现 2 个输入, 分别为 <i>clk</i>, <i>rst</i>, 分别连接时钟和复位信号; 需实现 2 个输出, 分别为 <i>pc</i>, <i>inst_ce</i>, 分别连接指令存储器的 <i>addra</i>, <i>ena</i> 端口。其中 <i>addra</i> 位数依据 coe 文件中指令数定义; 2. 加法器。用于计算下一条指令地址, 需实现 2 个输入, 1 个输出, 输入值分别为当前指令地址 <i>PC</i>、$32' h4$; 3. Controller。其中包含两部分: (a). <i>main_decoder</i>。负责判断指令类型, 并生成相应的控制信号。需实现 1 个输入, 为指令 <i>inst</i> 的高 6 位 <i>op</i>, 输出分为 2 部分, 控制信号有多个, 可作为多个输				

报告创建时间:

出，也作为一个多位输出，具体参照参考指导书进行设计；*aluop*，传输至 *alu_decoder*，使 *alu_decoder* 配合 *inst* 低 6 位 *funct*，进行 ALU 模块控制信号的译码。

(b). *alu_decoder*。负责 ALU 模块控制信号的译码。需实现 2 个输入，1 个输出，输入分别为 *funct*，*aluop*；输出位 *alucontrol* 信号。

(c). 除上述两个组件，需设计 *controller* 文件调用两个 *decoder*，对应实现 *op*，*funct* 输入信号，并传入调用模块；对应实现控制信号及 *alucontrol*，并连接至调用模块相应端口。

4. 指令存储器。使用 Block Memory Generator IP 构造。(参考指导书)

5. 时钟分频器。将板载 100Mhz 频率降低为 1hz，连接 PC、指令存储器时钟信号 *clk*。(参考数字逻辑实验)

三、实验过程或算法（源程序）

模块结构与解释

号	模块名称	描述
1.	Top2. v	把所有模块联合起来，要定义 <i>clk</i> 与 <i>rst</i>
2.	--clk_div.	把 200mhz 的 <i>clk</i> 周期分频到 1hz <i>clk</i> 周期
3.	--program_counter. v	每个周期要运行下个指令，于是定义在下个周期 <i>pc</i> 指令地址要加 4
4.	--pc_adder. v	特意为了 <i>pc</i> 每周期要加 4 的简单加法器
5.	--blk_mem_gen_0	本实验要调用的 ip 模块，模块连接到提供的. coe 文件。该文件里面有 6 个指令
6.	--controller. v	控制器模块，要跟据
7.	----main_decoder. v	要根据指令的操作码 <i>inst</i> [31:26] 决定要运行哪一类型指令，与不同的指令也有不同的控制信号需求
8.	----alu_decoder. v	处理 <i>alu</i> 部件需要运行哪个运算（比如：add, sub, or, and）
9.	--display. v	把 <i>pc_current</i> 的信号显示到开发板的七段数码显示器
10.	----seg7. v	要定义每个十六进制的值需要那些灯会亮

控制器接口定义（controller 模块的信号）

信号名	方向	位宽	功能描述
instr	输入	6	指令的操作码，来自 instr[31:26]。 操作码决定要执行哪个指令类型
instr_valid	输入	1	因为 ram 有延时，需要检查当前的指令 为有效的指令
jump	输出	1	若信号为 1，要跳转到新 pc 地址
branch	输出	1	若信号为 1，要运行 beq 指令
alu_src	输出	1	用于在有立即数运算
mem_write	输出	1	写存储（用于在 sw 指令）
mem_to_reg	输出	1	存储到寄存器（用于在 lw 指令）
reg_write	输出	1	写到寄存器
reg_dst	输出	1	写到寄存器的对象
func	输出	6	R 型指令辅助操作码
alu_control	输出	3	控制 alu 的运算（包括 add, sub, or, and）
alu_op	输出	2	Main_decoder 与 alu_decoder 之间的 alu 控制信号

指令分析

R 型

31-26	25-21	20-16	15-11	10-6	5-0
操作码	rs	rt	rd	shamt	funct

I 型

31-26	25-21	20-16	15-0
操作码	rs	rt	立即数

J 型

31-26	25-0
操作码	对象地址

```
shiyanke2 > coe > ASM inst_ram.asm
1  addi $2, $0, 5      # initialize $2=5      0  20020005
2  add $5, $5, $4      # $5 = 4 + 7 = 11     14 00a42820
3  lw $2, 80($0)       # $2 = [80] = 7       3c 8c020050
4  sw $2, 84($0)       # write adr 84=7      4c ac020054
5  beq $5, $7, end     # shouldn't be taken  18 10a7000c
6  j end              # should be taken      40 08000013
```

图 1：本实验提供的 RAM 指令文件

四、实验结果及分析和（或）源程序调试过程

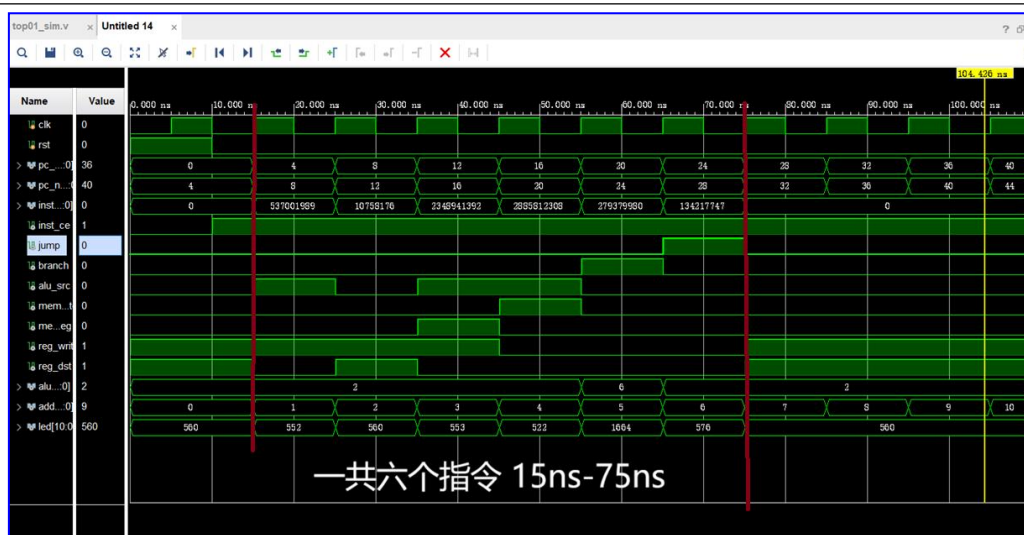


图 2: top.v 的仿真图, 由于 blockmemory 的局限有一个周期的延时

1. Top2.v

```
module top2 (
    input clk,
    input rst,
    output [10:0] led,      //为了测试控制部件的输出
    output [6:0] seg,      // 7 段数码管显示当前 pc 值
    output [7:0] ans       // 8 位数数码管显示当前 pc 值
);

//始终分频器信号
```

```

wire clk_slow;
//通过 clk 信号，生成一个 1 秒周期 clk_slow 信号
clk_div u_clk_div (
    .clk_in (clk),
    .rst_n (~rst),
    .clk_out(clk_slow)
);
wire [31:0] pc_current;      // pc 当前的值
wire [31:0] pc_next;        // pc 下个周期的值

wire [31:0] instruction;    // 来自 ram/rom 的指令（一共有 6 个指令）

// controller 模块的控制信号
wire jump;
wire branch;
wire alu_src;
wire mem_write;
wire mem_to_reg;
wire reg_write;
wire reg_dst;
wire [2:0] alu_control;

// ROM 地址线，需要把 pc_current 左移 2 位
wire [5:0] addra = pc_current[7:2];
wire inst_ce = ~rst;

// program_counter，处理 pc 地址更新，若有 rst 信号地址复制到 00000000
program_counter pc (
    .clk      (clk_slow),
    .rst      (rst),
    .inst_ce  (inst_ce),
    .pc_next  (pc_next),
    .pc_current (pc_current)
);

// program_counter，处理每个周期 pc 的地址会加 b'04
pc_adder u_pc_adder (
    .pc_current (pc_current),
    .pc_next    (pc_next)
);

// ROM，本实验提供的 rom 模块，
blk_mem_gen_0 ram (
    .clka  (clk_slow),
    .ena   (inst_ce),
    .addra (pc_current[7:2]),      // pc 的地址需要左移 2 位
    .douta (instruction)          // 32 位指令
);

// controller 某块，根据指令的操作码处理控制部件的输出信号
controller u_controller (
    .instr      (instruction[31:26]), // 操作码
    .func       (instruction[5:0]),   // r 型指令额外的操作码

```

```

        .jump      (jump),
        .branch    (branch),
        .alu_src    (alu_src),
        .mem_write  (mem_write),
        .mem_to_reg (mem_to_reg),
        .reg_write  (reg_write),
        .reg_dst    (reg_dst),
        .alu_control(alu_control)
    );
    //为了把 pc_current 的值显示在七段数码显示器
    display u_display (
        .clk(clk),
        .rst(rst),
        .s(pc_current),
        .seg(seg),
        .ans(ans)
    );
    //赋值 led 信号的值
    assign led[0] = mem_to_reg;
    assign led[1] = mem_write;
    assign led[2] = 0; //pc_src 的信号，但是本实验不会使用该信号
    assign led[3] = alu_src;
    assign led[4] = reg_dst;
    assign led[5] = reg_write;
    assign led[6] = jump;
    assign led[7] = branch;
    assign led[10:8] = alu_control;

```

endmodule

2. Clk_div.v

```

module clk_div(
    input    clk_in,
    input    rst_n,
    output reg clk_out
);

reg [26:0] cnt;
// 28 位的计数器
always @(posedge clk_in or negedge rst_n) begin
    if(!rst_n) begin
        cnt <= 27'd0;
    end
    else if(cnt == 27'd99999999) begin
        cnt <= 27'd0;
    end
    else begin
        cnt <= cnt + 1'b1;
    end
end
// 若计数器达到十进制 99999999，则周期翻转一次。一个周期位（1 秒）

```

```

always @(posedge clk_in or negedge rst_n) begin
    if(!rst_n) begin
        clk_out <= 1'b0;
    end
    else if(cnt == 27'd99999999) begin
        clk_out <= ~clk_out;
    end
end
endmodule

```

3. Program_counter.v

```

module program_counter (
    input clk,
    input rst,
    input [31:0] pc_next,
    output reg [31:0] pc_current,
    input inst_ce //指令使能信号，控制 pc 是否更新
);

//处理 pc 地址更新，若有 rst 信号则把 pc_current 重置为 0
always @(posedge clk or posedge rst) begin
    if (rst) begin
        pc_current <= 32'h00000000;
    end else if(inst_ce) begin
        pc_current <= pc_next;
    end
end
endmodule

```

4. Pc_adder.v

```

module pc_adder (
    input [31:0] pc_current, //当前指令地址
    output [31:0] pc_next //下个指令地址
);
//每次加 4，取下个指令的位置
assign pc_next = pc_current + 32'h4;
endmodule

```


6.controller.v

```
module controller (
    //main_decoder 的信号
    input [5:0] instr,          // instruct[31:26] = 操作码

    //控制信号
    output wire jump,
    output wire branch,
    output wire alu_src,
    output wire mem_write,
    output wire mem_to_reg,
    output wire reg_write,
    output wire reg_dst,

    //alu_decoder 的信号
    input [5:0] func,          // r 型指令的辅助操作码信号
    output wire [2:0] alu_control
);
    wire [1:0] alu_op;

    // Main decoder 的模块，根据指令的操作码生成控制部件的控制信号
    main_decoder u_main_decoder (
        .instr(instr),
        .jump(jump),
        .branch(branch),
        .alu_src(alu_src),
        .mem_write(mem_write),
        .mem_to_reg(mem_to_reg),
        .reg_write(reg_write),
        .reg_dst(reg_dst),
        .alu_op(alu_op)
    );

    // ALU decoder 的模块，根据 alu_op 与 func 生成 alu_control 的信号
    alu_decoder u_alu_decoder (
        .func(func),
        .alu_op(alu_op),
        .alu_control(alu_control)
    );
endmodule
```

7. Main_decoder.v

```

module main_decoder (
    input [5:0] instr,          // instruct[31:26] = 操作码

    //控制信号
    output reg jump,
    output reg branch,
    output reg alu_src,
    output reg mem_write,
    output reg mem_to_reg,
    output reg reg_write,
    output reg reg_dst,
    output reg [1:0] alu_op
);

always @(*) begin
    //默认值
    jump      = 1'b0;
    branch    = 1'b0;
    alu_src    = 1'b0;
    mem_write  = 1'b0;
    mem_to_reg = 1'b0;
    reg_write  = 1'b0;
    reg_dst    = 1'b0;
    alu_op     = 2'b00; //为 add 操作
    case(instr)
        //R 型指令
        6'b000000: begin
            reg_write = 1'b1;
            reg_dst   = 1'b1;
            alu_op    = 2'b10;
        end
        //lw
        6'b100011: begin
            reg_write = 1'b1;
            alu_src   = 1'b1;
            mem_to_reg = 1'b1;
        end
        //sw
        6'b101011: begin
            alu_src   = 1'b1;
            mem_write = 1'b1;
        end
        //beq
        6'b000100: begin
            branch    = 1'b1;
            alu_op    = 2'b01;
        end
        //addi
        6'b001000: begin
            reg_write = 1'b1;
            alu_src   = 1'b1;
        end
        //j
        6'b000010: begin
            jump      = 1'b1;
    endcase
end

```

```

        end
        default: begin
            //其他指令保持默认控制信号
        end
    endcase
end
endmodule

```

8. Alu_decoder.v

```

module alu_decoder (
    input [5:0] func,          // r 型指令的辅助操作码信号
    input [1:0] alu_op,        //来自 maindecoder
    output reg [2:0] alu_control //输出给 ALU 的控制信号
);

always @(*) begin
    case (alu_op)
        //lw/sw 指令
        2'b00: alu_control = 3'b010; //add
        //beq 指令
        2'b01: alu_control = 3'b110; //subtract
        //r 型指令，根据 func 字段选择 alu 的操作
        2'b10: begin
            case (func)
                6'b100000: alu_control = 3'b010; // add
                6'b100010: alu_control = 3'b110; // sub
                6'b100100: alu_control = 3'b000; // and
                6'b100101: alu_control = 3'b001; // or
                6'b101010: alu_control = 3'b111; // slt
                default: alu_control = 3'b010; // 默认为 add
            endcase
        end
        //默认为 add
        default: alu_control = 3'b010; // add
    endcase
end
endmodule

```

9. Display.v

```

module display(
    input wire clk,

```

```

input wire rst,
input wire [31:0]s,
output wire [6:0]seg,
output reg [7:0]ans
);

reg [20:0]count;
reg [4:0]digit;
//计数器模块
always@(posedge clk,posedge rst)
    if(rst)
        count = 0;
    else
        count = count + 1;
//扫描模块
always @(posedge clk)
    case(count[20:18])
        0:begin ans = 8'b11111110; digit = s[3:0]; end
        1:begin ans = 8'b11111101; digit = s[7:4]; end
        2:begin ans = 8'b11111011; digit = s[11:8]; end
        3:begin ans = 8'b11110111; digit = s[15:12]; end
        4:begin ans = 8'b11101111; digit = s[19:16]; end
        5:begin ans = 8'b11011111; digit = s[23:20]; end
        6:begin ans = 8'b10111111; digit = s[27:24]; end
        7:begin ans = 8'b01111111; digit = s[31:28]; end
    endcase
//调用七段数码显示器模块
seg7 U4(
    .din(digit),
    .dout(seg)
);
endmodule

```

10. 7seg.v

```

module seg7(

```

```

    input wire [3:0]din,      //十六进制输入
    output reg [6:0]dout
);

always@(*)
case(din)    //控制七段数码管显示的灯，一共有 7 个灯，信号位 1=灭，0=亮
    4'h0:dout = 7'b0000001; //显示 "0"
    4'h1:dout = 7'b1001111; //显示 "1"
    4'h2:dout = 7'b0010010; //显示 "2"
    4'h3:dout = 7'b0000110; //显示 "3"
    4'h4:dout = 7'b1001100; //显示 "4"
    4'h5:dout = 7'b0100100; //显示 "5"
    4'h6:dout = 7'b0100000; //显示 "6"
    4'h7:dout = 7'b0001111; //显示 "7"
    4'h8:dout = 7'b0000000; //显示 "8"
    4'h9:dout = 7'b0000100; //显示 "9"
    4'ha:dout = 7'b0001000; //显示 "a"
    4'hb:dout = 7'b1100000; //显示 "b"
    4'hc:dout = 7'b0110001; //显示 "c"
    4'hd:dout = 7'b1000010; //显示 "d"
    4'he:dout = 7'b0110000; //显示 "e"
    4'hf:dout = 7'b0111000; //显示 "f"
    default:dout = 7'b1111111; //显示空白
endcase
endmodule

```

备注：